



Organisation A

(Lead Backend & Sécurité Applicative)

Phase 1 & 2 (Semaines 1 à 3) : Construction du Cœur

Point de la Planification	Détails Techniques (Ce que Elod A doit apprendre et faire)
Apprendre les bases (Phase 1)	Java 21 et Spring Boot (injection de dépendances, structure MVC : Contrôleur/Service/Dépôt).
JWT & Hachage (Phase 2)	Apprendre à utiliser bcrypt ou Argon2 pour les mots de passe.
Tâches clés (Phase 2)	Implémenter l'émission du JWT signé (RS256 de préférence) via un filtre Spring Security personnalisé.

Phase 3 (Semaines 4 & 5) : Durcissement Applicatif

Point de la Planification	Détails Techniques (Ce que Elod A doit apprendre et faire)
Moindre privilège	Apprendre les annotations Spring Security pour les Autorisations par rôles sur les méthodes ou les points de terminaison.
Sondes de santé	Apprendre à configurer Spring Boot Actuator pour exposer les indicateurs (<code>/actuator/health/liveness</code> , etc.).
Qualité	Utiliser les annotations de validation Java (<code>@Valid</code>) pour durcir les données entrantes (Gestion d'erreurs propre).

Phase 4 (Semaines 6 & 7) : Hygiène et Finition

Point de la Planification	Détails Techniques (Ce que Elod A doit apprendre et faire)
Hygiène des dépendances	S'assurer que les dépendances (Spring, librairies JWT) sont à jour (aspect qualité et sécurité).
Secrets/Logs	Vérifier la configuration de la journalisation (Logger structuré) pour garantir qu'aucune information sensible (clés de signature, mots de passe) n'est écrite dans les logs.

Configuration Initiale d'IntelliJ IDEA (Personne A)

1. Extensions Essentielles (Plugins)

Allez dans **Settings** (ou **Preferences**) $\rightarrow$ **Plugins** et assurez-vous que les éléments suivants sont installés et activés :

- **Spring Assistant** : Ce plugin facilite la création de nouveaux projets Spring Boot. Il rend le processus de sélection des dépendances (comme **Spring Security**, **Spring Web**, **Spring Data JPA**) très rapide et intuitif. C'est un gain de temps majeur lors de l'initialisation.
- **Lombok Plugin** : Si vous utilisez la bibliothèque Lombok (très populaire en Java pour réduire le code répétitif comme les *getters*, *setters* et constructeurs), cette extension garantit qu'IntelliJ comprend bien les annotations de Lombok et ne signale pas d'erreurs.
- **EditorConfig** : Assure que le style de codage (indentation, retours à la ligne) est cohérent entre votre équipe et vos différents fichiers, un atout pour la qualité du code.

2. Configuration Spécifique à Spring Boot

Ces réglages vous aideront à gérer l'architecture de votre projet :

- **Navigation dans la Configuration** : Apprenez à utiliser la fonction de recherche avancée (double **Shift** ou **Ctrl + Shift + N** sur Windows/Linux, **Cmd + Shift + O** sur Mac) pour trouver rapidement les fichiers et les classes, notamment la classe de configuration principale de **Spring Security**.
- **Propriétés d'Application** : Familiarisez-vous avec le fichier **application.properties** ou **application.yml**. IntelliJ offre une bonne complétion automatique et une

coloration pour ces fichiers, qui seront cruciaux pour définir les secrets (**clés de signature JWT**) et les paramètres d'environnement.

3. Maîtrise du Débogueur (Crucial pour la Sécurité)

La partie la plus difficile sera de déboguer la **chaîne de filtres de Spring Security**. Vous devez absolument maîtriser l'outil de débogage.

- **Points d'Arrêt (Breakpoints)** : Apprenez à placer des points d'arrêt sur les méthodes clés de votre application :
 - La méthode de connexion (où le **JWT** est généré).
 - Le filtre de sécurité qui gère l'extraction et la validation du **JWT** dans l'en-tête `Authorization`.
 - Les points d'entrée (`@RestController`) qui appliquent les annotations de rôle (`@PreAuthorize`).
- **Examen des Variables** : Pendant le débogage, utilisez la fenêtre de variables pour inspecter le contenu du JWT (avant la signature) et l'état du **Contexte de Sécurité** de Spring (pour vérifier si l'utilisateur et ses rôles ont bien été ajoutés après la validation du jeton).

4. Outils Externes Intégrés

Utilisez IntelliJ pour exécuter vos commandes sans quitter l'environnement :

- **Fenêtre Terminal** : Utilisez le terminal intégré (souvent en bas de l'écran) pour lancer les commandes Maven ou Gradle (ex: `mvn spring-boot:run`) et pour exécuter les tests.
- **Version Control (Git)** : Assurez-vous que votre projet est correctement configuré avec Git et que vous utilisez les outils d'IntelliJ pour commettre, pousser et fusionner votre code avec le dépôt GitHub partagé avec la Personne B.

En optimisant ces outils dès le début, vous transformerez IntelliJ IDEA en un puissant allié pour surmonter la courbe d'apprentissage de Spring Security.